

Lesson 2 Structured Note: Array Stack Representation and Initialization

0. Current Position in the Course

You are now in **Lesson 2** of the stack curriculum.

This lesson belongs to:

Phase 1: Foundations and Array Stack

In Lesson 1, you learned the idea of a stack:

- A stack follows **LIFO**: Last In, First Out.
- Insertion and deletion happen only at the **top**.
- Basic stack operations include `push`, `pop`, `peek`, `stackTop`, `isEmpty`, and `isFull`.
- In an array stack, an empty stack is usually represented by `top = -1`.

Now Lesson 2 moves from the **idea** of a stack to the **memory representation** of a stack.

The main question in this lesson is:

How do we store a stack inside an array?

1. Lesson 2 Goal

By the end of this lesson, you should understand:

- What an **array stack** is.
 - Why an array stack needs `size`, `top`, and an array pointer.
 - What `top` means in an array stack.
 - Why `top` starts at `-1`.
 - How the first `push()` changes `top` from `-1` to `0`.
 - Why push and pop happen at the right side/end of the array.
 - Why pushing at the front of the array is inefficient.
 - What the empty condition is.
 - What the full condition is.
 - How to dynamically allocate stack memory in C++ and C.
 - The difference between stack **capacity** and current stack **content**.
 - The basic time and space complexity ideas for array stack initialization and operations.
-

2. What Is an Array Stack?

A stack is a data structure that follows this rule:

The last value inserted is the first value removed.

This rule is called:

LIFO: Last In, First Out

An **array stack** is a stack implemented using an array.

So instead of only thinking about the stack like this:

```
Top -> 30
       20
       10
```

We store it physically in an array like this:

```
Index:  0    1    2
Value: [10] [20] [30]
```

Here:

```
S[0] = 10
S[1] = 20
S[2] = 30
```

The top element is `30`, and its index is `2`.

So:

```
top = 2
```

Important idea:

In an array stack, `top` stores the index of the top element.

3. Three Things an Array Stack Needs

An array stack needs exactly three important fields:

```
int size;  
int top;  
int *S;
```

These are usually stored inside a `struct`.

```
struct Stack {  
    int size;  
    int top;  
    int *S;  
};
```

Meaning of Each Field

Field	Simple Meaning
<code>size</code>	Maximum number of elements the stack can store
<code>top</code>	Index of the current top element
<code>S</code>	Pointer to the array where stack values are stored

You can imagine the stack structure like this:

```
Stack  
├─ size = 5  
├─ top = -1  
└─ S -> array of 5 integers
```

4. Why Do We Need `size`?

An array has a fixed capacity.

For example, if the stack size is `5`, the array has 5 slots:

Index:	0	1	2	3	4
Value:	[]	[]	[]	[]	[]

This stack can store only 5 values.

So we need `size` to remember the maximum capacity of the stack.

Example:

```
size = 5;
```

The valid array indices are:

```
0, 1, 2, 3, 4
```

The last valid index is:

```
size - 1
```

For `size = 5`:

```
size - 1 = 4
```

So index `4` is the last valid position.

5. Why Do We Need `top`?

The variable `top` tells us where the current top element is.

Example:

Index:	0	1	2
Value:	[10]	[20]	[30]

Here, the top element is `30`.

`30` is stored at index `2`.

So:

```
top = 2;
```

This means:

```
The current top element is at S[2].
```

Important:

`top` does not store the value itself.

Wrong idea:

```
top = 30
```

Correct idea:

```
top = 2  
S[top] = 30
```

So `top` stores the **index**, not the actual stack value.

6. Why Do We Need `S`?

`S` is a pointer to the array that stores the stack elements.

Example:

```
int *S;
```

Later, we allocate memory for it:

```
S = new int[size];
```

or in C:

```
S = (int *)malloc(size * sizeof(int));
```

After allocation, `S` points to the first element of the array.

Example:

```
S -> [ ] [ ] [ ] [ ] [ ]
```

If `size = 5`, then `S` points to an array with 5 integer slots.

7. Empty State of an Array Stack

When an array stack is empty, we use:

```
top = -1;
```

This means:

```
There is no active element in the stack.
```

Example:

```
Index:  0    1    2    3    4
Value: [ ] [ ] [ ] [ ] [ ]

top = -1
```

The stack is empty because no valid array position is being used.

Important Warning

`top = -1` does not mean this exists:

```
S[-1]
```

Array index `-1` is invalid.

-1 is only a marker value.

It means:

No stack element exists yet.

8. Why Does `top` Start at -1?

Array indices start from 0.

The first valid array index is:

0

If the stack is empty, no valid index should be active.

So we need a value that is before index 0.

That value is:

-1

That is why an empty array stack starts with:

```
top = -1;
```

Then, during the first push, `top` moves from -1 to 0.

9. First Push Operation Concept

Suppose the stack is empty:

```
Index:  0    1    2    3    4
Value: [ ]  [ ]  [ ]  [ ]  [ ]

top = -1
```

Now we perform:

```
push(10);
```

A push operation has two main steps:

1. Increase `top`.
2. Store the new value at `S[top]`.

Step 1:

```
top++;
```

So:

```
top: -1 -> 0
```

Step 2:

```
S[top] = 10;
```

Since `top = 0`:

```
S[0] = 10;
```

Now the array becomes:

```
Index:  0    1    2    3    4
Value: [10] [ ] [ ] [ ] [ ]

top = 0
```

Logical stack view:

```
Top -> 10
```

Important result:

The first successful push changes `top` from `-1` to `0`.

10. Push More Values

Start after pushing 10:

```
Index:  0    1    2    3    4
Value: [10] [ ] [ ] [ ] [ ]

top = 0
```

Now perform:

```
push(20);
```

top increases:

```
top: 0 -> 1
```

Store 20:

```
S[1] = 20;
```

Array:

```
Index:  0    1    2    3    4
Value: [10] [20] [ ] [ ] [ ]

top = 1
```

Now perform:

```
push(30);
```

top increases:

```
top: 1 -> 2
```

Store 30:

```
S[2] = 30;
```

Array:

```
Index:  0    1    2    3    4
Value: [10] [20] [30] [ ] [ ]

top = 2
```

Logical stack view:

```
Top -> 30
       20
       10
```

11. Physical Array View vs Logical Stack View

This is an important beginner concept.

The physical array may look like this:

```
Index:  0    1    2
Value: [10] [20] [30]
```

But the logical stack view is:

```
Top -> 30
       20
       10
```

Why?

Because in stack logic, the newest value is at the top.

The array stores values from left to right:

```
10, 20, 30
```

But the stack is usually visualized from top to bottom:

```
30, 20, 10
```

Important idea:

The physical array order and the logical stack display order are not the same thing.

12. Why Push and Pop Happen at the Right Side/End

In an array stack, we use the right side/end of the used portion of the array as the top.

Example:

```
Index:  0      1      2      3      4
Value: [10] [20] [30] [ ] [ ]

top = 2
```

The top value is:

```
S[top] = S[2] = 30
```

If we push `40`, we do:

```
top++;
S[top] = 40;
```

Result:

```
Index:  0      1      2      3      4
Value: [10] [20] [30] [40] [ ]

top = 3
```

No existing values moved.

Only two simple actions happened:

1. Increase `top`.
2. Store the new value.

That is why push at the end is efficient.

Time complexity:

$O(1)$

13. Why We Do Not Push at the Front

Suppose the array has:

Index:	0	1	2
Value:	[10]	[20]	[30]

If we try to push `40` at index `0`, we must first shift the existing values.

Before shifting:

[10] [20] [30]

After shifting right:

[] [10] [20] [30]

Then insert `40` at the front:

[40] [10] [20] [30]

This takes more work because many values may need to move.

If there are `n` elements, we may need to shift `n` elements.

So the time complexity becomes:

$O(n)$

That is not good for stack operations.

A stack should normally perform push and pop in constant time:

$O(1)$

So we push and pop at the right side/end, where `top` is.

14. Pop Concept in an Array Stack

Although Lesson 2 mainly focuses on representation and initialization, pop is useful for understanding why `top` matters.

Suppose the stack is:

```
Index:  0      1      2
Value: [10]  [20]  [30]

top = 2
```

Logical view:

```
Top -> 30
      20
      10
```

If we perform:

```
pop();
```

The stack removes the top element.

The top element is:

```
S[top] = S[2] = 30
```

The pop operation usually does this:

1. Save `S[top]`.
2. Decrease `top`.
3. Return the saved value.

So:

```
Returned value = 30  
top: 2 -> 1
```

After pop:

```
Index:  0      1      2  
Value: [10]  [20]  [30]  
  
top = 1
```

Important memory note:

`30` may still physically remain in the array memory at index `2`.

But it is no longer logically part of the stack because `top = 1`.

The active stack is now:

```
Top -> 20  
      10
```

Index `2` is ignored.

15. Empty Condition

The stack is empty when:

```
top == -1
```

Example:

```
Index:  0    1    2    3    4
Value: [ ] [ ] [ ] [ ] [ ]

top = -1
```

This means:

There are no active elements in the stack.

If we try to pop from an empty stack, that is called:

Stack Underflow

Example:

```
if (top == -1) {
    cout << "Stack Underflow";
}
```

16. Full Condition

An array stack has fixed capacity.

If the stack size is , valid indices are:

0, 1, 2, 3, 4

The last valid index is:

size - 1

So the stack is full when:

top == size - 1

For example:

```
size = 5;
```

Full condition:

```
top == 4
```

Example full stack:

```
Index:  0      1      2      3      4
Value: [10]  [16]  [20]  [25]  [30]

top = 4
```

If we try to push another value, there is no valid slot left.

That is called:

Stack Overflow

Example:

```
if (top == size - 1) {
    cout << "Stack Overflow";
}
```

17. Empty and Full Formulas

These two formulas are very important.

Empty Formula

```
top == -1
```

Meaning:

```
The stack has no active elements.
```


Full Formula

```
top == size - 1
```

Meaning:

```
The stack is using the last valid array index.
```

18. Capacity vs Current Number of Elements

Do not confuse `size` and `top`.

Example:

```
size = 5;  
top = 2;
```

This does not mean the stack has size 2.

It means:

```
The stack capacity is 5.  
The current top element is at index 2.
```

Active indices are:

```
0, 1, 2
```

So the current number of active elements is:

```
3
```

In general, when the stack is not empty:

```
number of elements = top + 1
```

Example:

```
top = 2
```

So:

```
number of elements = 2 + 1 = 3
```

Important distinction:

Variable	Meaning
size	Total capacity
top	Index of current top element
top + 1	Current number of active elements

19. Static Size vs Dynamic Allocation

There are two broad ways to create an array for a stack.

19.1 Fixed Array Size

You could write:

```
int S[5];
```

This creates an array of 5 integers.

But the size is fixed in the code.

If the user wants 10 elements, the code must be changed.

19.2 Dynamic Allocation

A better method for this lesson is to ask the user for the stack size at runtime.

Example:

```
cout << "Enter stack size: ";  
cin >> st.size;
```

Then allocate memory:

```
st.S = new int[st.size];
```

This creates the array while the program is running.

If the user enters , the program creates 5 slots.

If the user enters , the program creates 10 slots.

That is called:

Dynamic memory allocation

20. C++ Array Stack Structure

In C++, the stack structure can be written like this:

```
struct Stack {  
    int size;  
    int top;  
    int *S;  
};
```

Meaning:

```
int size;
```

Stores the maximum capacity.

```
int top;
```

Stores the index of the top element.

```
int *S;
```

Points to the dynamically allocated array.

21. C++ Initialization Example

```
#include <iostream>
using namespace std;

struct Stack {
    int size;
    int top;
    int *S;
};

int main() {
    Stack st;

    cout << "Enter stack size: ";
    cin >> st.size;

    st.S = new int[st.size];
    st.top = -1;

    cout << "Stack created successfully." << endl;
    cout << "Size: " << st.size << endl;
    cout << "Top: " << st.top << endl;

    delete[] st.S;

    return 0;
}
```

Explanation of the Code

```
Stack st;
```

Creates a stack variable named `st`.

```
cin >> st.size;
```

Reads the stack size from the user.

```
st.S = new int[st.size];
```

Allocates an integer array on the heap.

```
st.top = -1;
```

Marks the stack as empty.

```
delete[] st.S;
```

Releases the dynamically allocated memory.

22. Why Set `top = -1` Immediately After Allocation?

After memory is allocated, the array exists.

But it does not yet contain active stack elements.

Example:

```
st.S = new int[st.size];  
st.top = -1;
```

The memory exists:

Index:	0	1	2	3	4
Value:	[]	[]	[]	[]	[]

But the stack is empty:

```
top = -1
```

Important idea:

Allocating memory does not mean the stack has elements.

The array has capacity, but no active stack values yet.

23. C-Style Array Stack Structure

In C, the structure is similar:

```
struct Stack {  
    int size;  
    int top;  
    int *S;  
};
```

The difference is mainly how memory is allocated.

In C, we usually use:

```
malloc()
```

and release memory using:

```
free()
```

24. C Initialization Example

```
#include <stdio.h>  
#include <stdlib.h>  
  
struct Stack {  
    int size;  
    int top;  
    int *S;
```

```
};

int main() {
    struct Stack st;

    printf("Enter stack size: ");
    scanf("%d", &st.size);

    st.S = (int *)malloc(st.size * sizeof(int));
    st.top = -1;

    printf("Stack created successfully.\n");
    printf("Size: %d\n", st.size);
    printf("Top: %d\n", st.top);

    free(st.S);

    return 0;
}
```

Explanation of the C Code

```
struct Stack st;
```

Creates a stack variable named `st`.

```
scanf("%d", &st.size);
```

Reads the stack size from the user.

```
st.S = (int *)malloc(st.size * sizeof(int));
```

Allocates enough heap memory for `st.size` integers.

```
st.top = -1;
```

Marks the stack as empty.

```
free(st.S);
```

Releases the allocated heap memory.

25. Meaning of

```
malloc(st.size * sizeof(int))
```

In C, memory allocation is written like this:

```
malloc(st.size * sizeof(int))
```

Suppose:

```
st.size = 5;
```

Then:

```
st.size * sizeof(int)
```

means:

```
5 times the memory needed for one integer
```

So the program allocates enough space for 5 integers.

The cast:

```
(int *)
```

means:

```
Treat the returned memory address as a pointer to int.
```

So:

```
st.S = (int *)malloc(st.size * sizeof(int));
```

means:

Allocate memory for the stack array and store its address in `st.S`.

26. Required Dry Run: User Enters Size 5

Suppose the user enters:

```
5
```

Then:

```
size = 5
```

The program creates 5 slots:

```
Index:  0    1    2    3    4  
Value: [ ]  [ ]  [ ]  [ ]  [ ]
```

Then it sets:

```
top = -1
```

So the current state is:

```
size = 5  
top = -1  
Stack is empty
```

Now suppose the first operation is:

```
push(10);
```

The first push does this:

```
top++;  
S[top] = 10;
```

Step-by-step:

```
top moves from -1 to 0  
S[0] becomes 10
```

Result:

```
Index:  0    1    2    3    4  
Value: [10] [ ] [ ] [ ] [ ]  
  
top = 0
```

27. Full Condition Dry Run for Size 5

Given:

```
size = 5;
```

Valid indices:

```
0, 1, 2, 3, 4
```

Full condition:

```
top == size - 1
```

Substitute `size = 5`:

```
top == 5 - 1
```

So:

```
top == 4
```

When `top = 4`, the stack is full.

Example:

```
Index:  0      1      2      3      4
Value: [10] [20] [30] [40] [50]
```

```
top = 4
```

Trying to push another value would cause overflow.

28. Why Array Stacks Are Fixed Capacity

When you allocate an array like this:

```
st.S = new int[st.size];
```

or:

```
st.S = (int *)malloc(st.size * sizeof(int));
```

The stack gets a fixed number of slots.

Example:

```
size = 5
```

The stack has exactly 5 slots:

```
0, 1, 2, 3, 4
```

After these slots are filled, the stack is full.

The stack cannot automatically grow unless we add extra reallocation logic.

That extra resizing logic is outside this lesson.

So for this lesson:

An array stack has fixed capacity after creation.

29. Time Complexity Notes

Initialization

Creating the stack involves:

1. Reading the size.
2. Setting `top = -1`.
3. Allocating an array of `n` slots.

The control work is simple:

$O(1)$

But the storage allocated is proportional to the size of the stack:

$O(n)$

So:

Action	Complexity
Set <code>top = -1</code>	$O(1)$
Store <code>size</code>	$O(1)$
Allocate array of <code>n</code> slots	$O(n)$ space

Push and Pop at the End

Push and pop at the top/end do not require shifting.

They only update `top` and access one array position.

So each operation is:

$O(1)$

Pushing at the Front

If we push at the front, we must shift existing values.

That can take:

$O(n)$

That is why array stacks use the end side as the top.

30. Space Complexity Notes

The stack array itself takes space based on its capacity.

If the stack size is `n`, the array uses:

$O(n)$

But individual operations like push and pop use only a few extra variables.

So their auxiliary space is:

$O(1)$

Important distinction:

Item	Space
Stack storage array	$O(n)$
One push operation	$O(1)$ auxiliary space
One pop operation	$O(1)$ auxiliary space

31. Important Beginner Mistakes

Mistake 1: Thinking `top = -1` Is a Real Array Index

Wrong:

`S[-1]` exists

Correct:

```
-1 only means the stack is empty.
```

Array index `-1` is invalid.

Mistake 2: Starting an Empty Stack with `top = 0`

Wrong initialization:

```
top = 0;
```

This makes it look like index `0` is already active.

Correct initialization:

```
top = -1;
```

Then the first push moves `top` to `0`.

Mistake 3: Thinking `top` Stores the Top Value

Wrong:

```
top = 30
```

Correct:

```
top = 2  
S[top] = 30
```

`top` stores the index of the top value.

Mistake 4: Confusing `size` and `top`

If:

```
size = 5;  
top = 2;
```

Wrong idea:

The stack size is 2.

Correct idea:

The stack capacity is 5.
The top element is at index 2.
There are 3 active elements.

Mistake 5: Forgetting the Full Formula

Wrong:

```
top == size
```

Correct:

```
top == size - 1
```

Why?

Because array indexing starts at 0.

If `size = 5`, the last valid index is 4, not 5.

Mistake 6: Thinking Allocated Memory Means Active Elements Exist

After allocation:

```
st.S = new int[5];
```

The array exists.

But the stack is still empty until values are pushed.

That is why we set:

```
st.top = -1;
```

32. Key Formulas and Rules

Array Stack Fields

```
int size;  
int top;  
int *S;
```

Empty Condition

```
top == -1
```

Full Condition

```
top == size - 1
```

First Valid Index

```
0
```

Last Valid Index

```
size - 1
```

First Push Effect

```
top: -1 -> 0
```


Push at Top/End

```
top++;  
S[top] = value;
```

Number of Active Elements

When the stack is not empty:

```
number of elements = top + 1
```

33. Mini Trace Practice 1

Given:

```
size = 5  
top = -1
```

Perform:

```
push(10)  
push(20)  
push(30)
```

Step-by-step:

Step	Operation	top Before	top After	Active Stack Values
0	Start	-1	-1	Empty
1	push(10)	-1	0	10
2	push(20)	0	1	10, 20
3	push(30)	1	2	10, 20, 30

Final array:

```
Index:  0    1    2    3    4  
Value: [10] [20] [30] [ ] [ ]
```

```
top = 2
```

Logical stack:

```
Top -> 30
      20
      10
```

34. Mini Trace Practice 2

Given:

```
size = 5
top = 2
```

Array:

Index:	0	1	2	3	4
Value:	[10]	[20]	[30]	[]	[]

Question:

Is the stack full?

Full condition:

```
top == size - 1
```

Substitute:

```
2 == 5 - 1
```

```
2 == 4
```

This is false.

So the stack is not full.

There are still empty slots at indices `3` and `4`.

35. Mini Trace Practice 3

Given:

```
size = 5
top  = 4
```

Question:

Is the stack full?

Full condition:

```
top == size - 1
```

Substitute:

```
4 == 5 - 1
```

```
4 == 4
```

This is true.

So the stack is full.

Another push would cause stack overflow.

36. Checkpoint Questions

Try to answer these without looking at the answers.

1. What are the three fields needed for an array stack?
2. What does `size` store?
3. What does `top` store?

4. What does `S` store?
 5. Why is `top` initialized to `-1`?
 6. Does array index `-1` exist?
 7. If `size = 5`, what are the valid indices?
 8. If `size = 5`, what is the full condition?
 9. If `size = 6`, what is the full condition?
 10. What happens to `top` during the first push?
 11. Why do we push and pop at the right side/end of the array?
 12. What would happen if we pushed at the front of the array?
 13. What is stack overflow?
 14. What is stack underflow?
 15. What is the difference between capacity and current number of elements?
-

37. Checkpoint Answers

1. `size`, `top`, and `S`.
 2. `size` stores the maximum number of elements the stack can hold.
 3. `top` stores the index of the current top element.
 4. `S` stores the address of the array used for stack storage.
 5. Because no valid array index is active when the stack is empty.
 6. No. `-1` is only a marker for empty.
 7. `0, 1, 2, 3, 4`.
 8. `top == 4`.
 9. `top == 5`.
 10. `top` moves from `-1` to `0`.
 11. Because pushing and popping at the end does not require shifting, so it is $O(1)$.
 12. Existing values would need to shift, making the operation $O(n)$.
 13. Stack overflow means trying to push into a full stack.
 14. Stack underflow means trying to pop from an empty stack.
 15. Capacity is the maximum size. Current number of elements is how many values are active now.
-

38. Practice Tasks

Task 1

Draw an empty array stack of size `6`.

Label:

- All valid indices.
- The value of `top`.
- The empty condition.

Expected ideas:

```
Index:  0    1    2    3    4    5
Value: [ ]  [ ]  [ ]  [ ]  [ ]  [ ]

top = -1
```

Task 2

Write the `struct Stack` definition in C++.

Expected answer:

```
struct Stack {
    int size;
    int top;
    int *S;
};
```

Task 3

Write the `struct Stack` definition in C style.

Expected answer:

```
struct Stack {
    int size;
    int top;
    int *S;
};
```

Task 4

Explain what would go wrong if `top` started at `0` in an empty stack.

Expected idea:

The program may think index 0 already contains an active stack element.
The correct empty value is `top = -1`.

Task 5

Given:

```
size = 7
```

Answer:

1. What is the first valid index?
2. What is the last valid index?
3. What is the empty condition?
4. What is the full condition?

Expected answers:

```
First valid index: 0
Last valid index: 6
Empty condition: top == -1
Full condition: top == 6
```

39. Short Glossary

Term	Simple Meaning
Stack	A data structure that follows LIFO
LIFO	Last In, First Out
Array stack	A stack implemented using an array
<code>size</code>	Maximum capacity of the stack
<code>top</code>	Index of the current top element
<code>S</code>	Pointer to the array storage
Empty stack	A stack with no active elements
Full stack	A stack with no free slots left

Term	Simple Meaning
Overflow	Trying to push into a full stack
Underflow	Trying to pop from an empty stack
Dynamic allocation	Creating memory at runtime
<code>new</code>	C++ operator used to allocate memory
<code>delete[]</code>	C++ operator used to release dynamic arrays
<code>malloc</code>	C function used to allocate memory
<code>free</code>	C function used to release memory
Capacity	Maximum number of values the stack can hold
Active elements	Values currently inside the stack

40. Final Lesson 2 Summary

Lesson 2 teaches how to represent a stack using an array.

An array stack needs three fields:

```
int size;
int top;
int *S;
```

`size` stores the maximum capacity.

`top` stores the index of the current top element.

`S` points to the array where the stack values are stored.

When the stack is empty:

```
top == -1
```

When the stack is full:

```
top == size - 1
```

The first push changes:

top: -1 -> 0

Push and pop happen at the right side/end of the array because this avoids shifting values.

That keeps stack operations efficient:

$O(1)$

Pushing at the front would require shifting elements, which would take:

$O(n)$

An array stack has fixed capacity after creation unless extra resizing logic is added.

The most important idea to remember is:

`top` stores the index of the current top element, not the value itself.

Another important idea:

`top = -1` means the stack is empty. It does not mean array index `-1` exists.

If you understand these points clearly, you are ready for Lesson 3, where you will implement actual array stack operations such as `push`, `pop`, `peek`, `stackTop`, `isEmpty`, and `isFull`.